AI Incident Command Center

A real-time AI-powered incident management platform built using React, Express, PostgreSQL, Prisma, Socket.IO, and Groq AI.

This application helps teams report, track, and manage live incidents collaboratively while receiving AI-generated incident summaries and suggested next actions.

Live Features

- Real-time incident updates
- AI-generated incident summaries
- Suggested next actions using AI
- Incident status workflow
- PostgreSQL database integration
- Socket.IO realtime synchronization
- Prisma ORM integration
- Responsive modern UI
- Dashboard auto-refresh without page reload

Tech Stack

Frontend

- React
- Vite
- Tailwind CSS
- Axios
- React Router DOM
- Socket.IO Client

Backend

- Node.js
- Express.js
- Socket.IO
- Prisma ORM
- OpenAI SDK

Database

- PostgreSQL
- pgAdmin

AI Integration

- Groq API
- Llama 3.1 Model

Project Structure

```
ai-incident-room/
│
├── client/
│   ├── src/
│   │   ├── pages/
│   │   ├── services/
│   │   └── App.jsx
│
├── server/
│   ├── prisma/
│   ├── server.js
│   └── .env
```

Complete Setup Guide

1. Clone Repository

```
git clone YOUR_GITHUB_REPOSITORY_URL
```

```
cd ai-incident-room
```

---

2. Frontend Setup

Open terminal:

```
cd client
```

Install frontend dependencies:

```
npm install
```

Run frontend:

```
npm run dev
```

Frontend runs on:

```
http://localhost:5173
```

---

3. Backend Setup

Open another terminal:

```
cd server
```

Install backend dependencies:

```
npm install
```

---

4. PostgreSQL Setup

Open pgAdmin.

Create database:

```
ai_incident_room
```

Default PostgreSQL details:

```
Username: postgres
Port: 5432
```

---

5. Configure Environment Variables

Inside:

```
server/.env
```

Add:

```
DATABASE_URL="postgresql://postgres:YOUR_PASSWORD@localhost:5432/
ai_incident_room"

PORT=5000

GROQ_API_KEY=YOUR_GROQ_API_KEY
```

Example:

```
DATABASE_URL="postgresql://postgres:komal123@localhost:5432/ai_incident_room"

PORT=5000

GROQ_API_KEY=gsk_xxxxxxxxxxxxxxxxx
```

---

6. Prisma Setup

Initialize Prisma migration:

```
npx prisma migrate dev --name init
```

Generate Prisma client:

```
npx prisma generate
```

---

7. Start Backend Server

Inside server folder:

```
npm run dev
```

Backend runs on:

```
http://localhost:5000
```

---

8. Test Backend API

Open browser:

```
http://localhost:5000/incidents
```

Expected output:

```
[]
```

---

9. Realtime Architecture

Realtime communication implemented using Socket.IO.

Flow:

```
User adds update
→ Backend stores update in PostgreSQL
→ Socket.IO emits realtime event
→ Connected clients receive update instantly
→ Dashboard refreshes automatically
```

---

10. AI Integration

This project uses:

- Groq API
- Llama 3.1 8B Instant model

AI functionality:

- Incident summarization
- Suggested next actions

To use AI feature:

1. Create Groq account
2. Generate API key
3. Add key inside `.env`
4. Restart backend server

---

11. Features Overview

Incident Dashboard

- View active incidents
- Priority badges
- Status badges
- Latest updates
- Created timestamps

Incident Details

- Add updates
- View incident timeline
- Change incident status
- Generate AI summaries

Status Workflow

Supported statuses:

- Open
- Investigating
- Resolved

Resolved incidents automatically disappear from dashboard.

---

12. Database Schema

Incident

- id
- title
- description
- priority
- status
- reporter_name
- latest_update
- created_at
- updated_at

IncidentUpdate

- id
- incident_id
- message
- author_name
- created_at

AIResult

- id
- incident_id
- type
- result_text
- created_at

---

13. Deployment

Frontend Deployment

- Vercel

Backend Deployment

- Render

Database Hosting

- Neon PostgreSQL

---

14. Future Improvements

- User authentication
- Role-based access
- Search and filtering
- File uploads
- Notifications
- Analytics dashboard
- Dark mode

Author

Komal Mathamsetti

License